Trenažieru zāles grafika optimizācija

Praktiskā kombinatoriālā optimizācija · Latvijas Universitāte
Gunārs Ābeltiņš · studenta apliecības Nr. **ga26014**

1. Uzdevuma formulējums

Ar pasniedzēju saskaņotais uzdevums ir treniņu plānošana ar **fiksētu vingrinājumu secību**. Visi cilvēki ierodas laikā 0. Katram vingrinājumam dots konkrēts aprīkojuma eksemplārs un pozitīvs vesels ilgums minūtēs. Cilvēks un iekārta vienlaikus piedalās tikai vienā vingrinājumā; tas netiek pārtraukts. Dažādas iekārtas var lietot paralēli. Atpūtu, pārvietošanos un sagatavošanu ignorē.

Mērķis ir minimizēt **visu cilvēku kopējo gaidīšanas laiku**, ieskaitot gaidīšanu pirms pirmā vingrinājuma un starp vingrinājumiem:

$$W = \sum_{i=1}^{n} \left(C_i - \sum_{j=1}^{m_i} p_{ij} \right),$$

Te C_i ir treniņa pabeigšanas laiks, m_i ir vingrinājumu skaits un p_{ij} ir to ilgumi. Pēdējā cilvēka pabeigšanas laiku (**makespan**) ziņo atsevišķi, bet neoptimizē.

2. Algoritma apraksts

2.1. Domēns

Risinājums ir cilvēku identifikatoru saraksts: katrs cilvēks parādās vienu reizi par katru savu vingrinājumu. Parādīšanās vienmēr nozīmē viņa **nākamo** vingrinājumu. Piemēram, `[0, 1, 0]` ieplāno cilvēka 0 pirmo, cilvēka 1 pirmo un cilvēka 0 otro vingrinājumu. Fiksētā secība saglabājas.

2.2. Novērtēšana

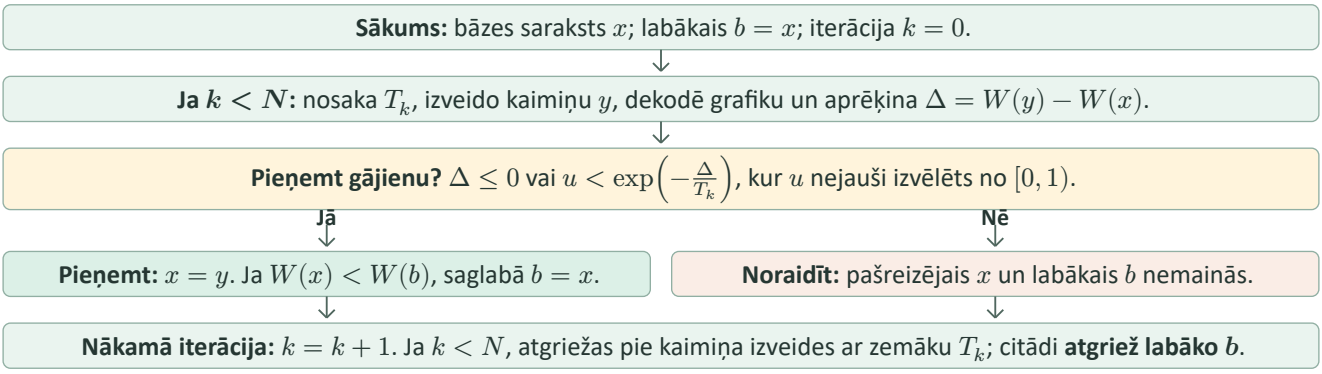
Katru vingrinājumu sāk, tiklīdz brīvs gan cilvēks, gan iekārta. To **pievieno aiz pēdējā ieplānotā vingrinājuma** uz šīs iekārtas; agrākos brīvos intervālos to neievieto. To izmantošanai jāmaina prioritāšu saraksts. No iegūtā grafika aprēķina kopējo gaidīšanu un pēdējās pabeigšanas laiku.

2.3. Gājiens

Nejauši izvēlas divas pozīcijas ar atšķirīgiem cilvēku identifikatoriem. Ar vienādu varbūtību tos samaina vai vienu izņem un ievieto otras pozīcijā. Identifikatoru skaits saglabājas, tāpēc arī jaunais grafiks ir pieļaujams.

2.4. Algoritms

Izmantota pašrocīgi implementēta **simulētā atkvēlināšana** (SA) Python vidē; gatavas optimizācijas bibliotēkas nav izmantotas. Bāzes sarakstā vispirms ir visu cilvēku pirmie vingrinājumi ievades secībā, tad otrie utt. Diagrammā x ir pašreizējais, y ir kaimiņš un b ir labākais risinājums. Sliktāku gājienu reizēm pieņem, lai izkļūtu no lokāla minimuma. Temperatūru ģeometriski samazina no 50 līdz 0,1; beigās atgriež labāko grafiku. Eksperimentos ir 1000 vai 5000 iterācijas, un nejaušību kontrolē sēkla.



3. Testēšanas apraksts

Izveidoti 14 JSON piemēri: 6 mazi un 8 lielāki ar 5, 10, 20 un 30 cilvēkiem, katram 3 līdz 8 vingrinājumiem. Katrā lielumā ir divi noslodzes varianti. Katru piemēru testē ar sēklām 0 līdz 4 un abiem iterāciju budžetiem: kopā **140 SA izpildes**, 14 bāzes grafiki un 6 precīzās meklēšanas izpildes. Laiku mēra ar `time.perf_counter()`, neiekļaujot pārbaudes un failu saglabāšanu.

3.1. Precīzā meklēšana un pareizības pārbaudes

Maziem piemēriem pašrocīgi pārmeklē visus prioritāšu sarakstus. Zaru atmet, ja jau uzkrātā gaidīšana sasniedz labāko zināmo vērtību: turpinājumā tā nevar samazināties.

Meklēšana aptver optimumu: jebkura pieļaujama grafika cilvēku un iekārtu secības veido aciklisku precedences grafu. Tā agrākais izpildījums gaidīšanu nepalielina, un topoloģisko secību var atveidot ar prioritāšu sarakstu. Precīzo meklēšanu veic līdz 10 vingrinājumiem, ar 10 sekunžu un 1 000 000 mezglu limitu. Pārtrauktu meklēšanu neatzīmē kā optimālu.

Mazais piemērs	Bāze, min	Optimums, min	Labākais SA, min
Viens cilvēks	0	0	0
Bez konfliktiem	0	0	0
Viena iekārta: 3 un 1 min	3	1	1
Krustota iekārtu secība	0	0	0
Atkārtota iekārtas lietošana	2	1	1
Trīs cilvēki	5	3	3

Visi seši optimumi ir pierādīti; SA tos sasniedza katrā sēklā un budžetā. Izturēti 8 testi par secību, ilgumiem, pārklāšanos, pilnīgumu, gaidīšanu, atkārtojamību un limitiem. Precīzais solveris sakrita ar neatkarīgu iekārtu secību pārmeklēšanu 14 mazos piemēros. Pārbaudīts arī katrs eksperimenta grafiks.

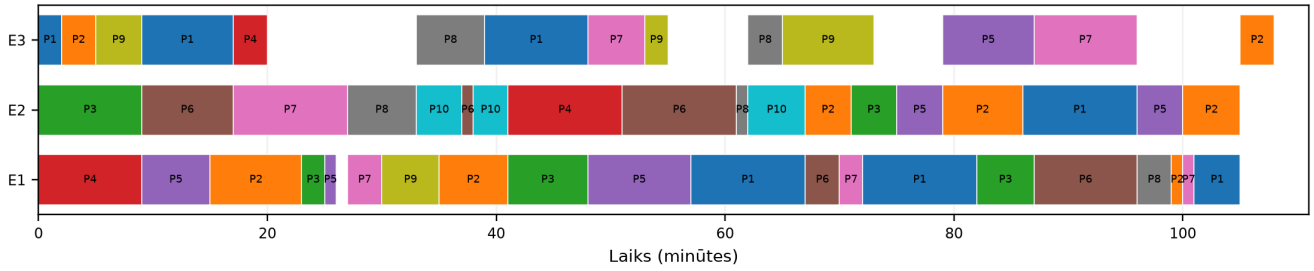
3.2. Lielāko piemēru rezultāti

Cilvēki / noslodze	Iek. / vingr.	Bāze	SA 1000 labākais	SA 5000 labākais	SA 5000 vidējais	SA 5000 laiks, s
5 / augsta	2 / 26	345	118	111	118,0	0,298
5 / zema	8 / 24	21	14	14	14,0	0,283
10 / augsta	3 / 52	605	367	337	377,2	0,466
10 / zema	8 / 57	305	143	130	142,2	0,538
20 / augsta	3 / 113	2904	2182	1944	2024,2	1,005
20 / zema	10 / 126	1072	758	667	697,6	1,210
30 / augsta	4 / 171	5304	4041	3324	3606,6	1,580
30 / zema	12 / 173	2576	1580	1355	1395,8	1,622

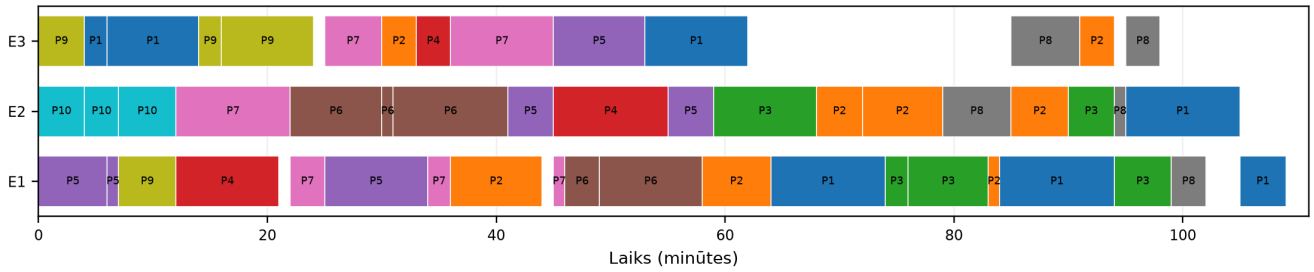
Gaidīšanas vērtības ir minūtēs, bet izpildes laiks ir piecu SA izpilžu vidējais sekundēs. Lielo piemēru optimumi nav pierādīti; tabulā norādīti labākie novērotie rezultāti. 5000 iterācijas uzlaboja labāko rezultātu 7 no 8 lielajiem piemēriem; `gym_05_low` labākā vērtība palika 14.

4. Grafiku salīdzinājums un secinājumi

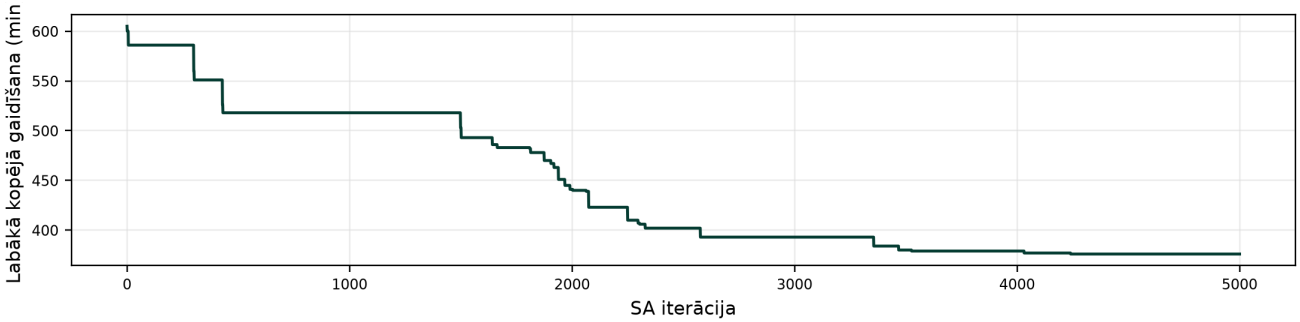
Piemērs `gym_10_high`: 10 cilvēki, 3 iekārtas. Abu grafiku laika mērogs ir vienāds; krāsa un apzīmējums `P1`, `P2` utt. norāda cilvēku.



Attēls 1: Bāzes grafiks: kopējā gaidīšana 605 min, pēdējā pabeigšana 108 min.



Attēls 2: SA grafiks: sēkla 2, 5000 iterācijas; gaidīšana 376 min, pēdējā pabeigšana 109 min.



Attēls 3: Labākās kopējās gaidīšanas izmaiņas SA izpildes laikā (sēkla 2).

Gaidīšana samazinās par **37,9 %**, lai gan pēdējais cilvēks pabeidz minūti vēlāk: abi kritēriji ir atšķirīgi. Attēlos ir sēkla 2; labākais rezultāts piecās izpildēs ir 337 min, bet vidējais ir 377,2 min.

SA uzlaboja bāzi visos astoņos lielajos piemēros. Pie 30 cilvēkiem un augstas noslodzes labākā gaidīšana samazinājās no 5304 līdz 3324 min (**37,3 %**). 5000 iterācijas parasti uzlaboja kvalitāti, bet prasīja ilgāku izpildi. Vienādi temperatūras galapunkti nozīmē arī atšķirīgu dzesēšanas ātrumu.

Ierobežojumi: piecas sēklas ir neliels salīdzinājums; temperatūra nav pielāgota katram piemēram. Noslodzes variantos atšķiras arī treniņi, tāpēc iekārtu skaita ietekme nav izolēta. Lielo piemēru SA rezultātiem nav optimalitātes garantijas.

5. Programmas kods un testa piemēri

GitHub repozitorijs: https://github.com/GunarsA/LU-PKO-Md1

Repozitorijā ir Python kods, JSON testa piemēri, CSV mērījumi un palaišanas instrukcija. Eksperimentus atkārto ar `python main.py experiments`; pilnie mērījumi ir `results/experiments.csv`.